

Java Backend Architecture

Interview Series

Chapter 10.4

Elasticsearch Query DSL

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

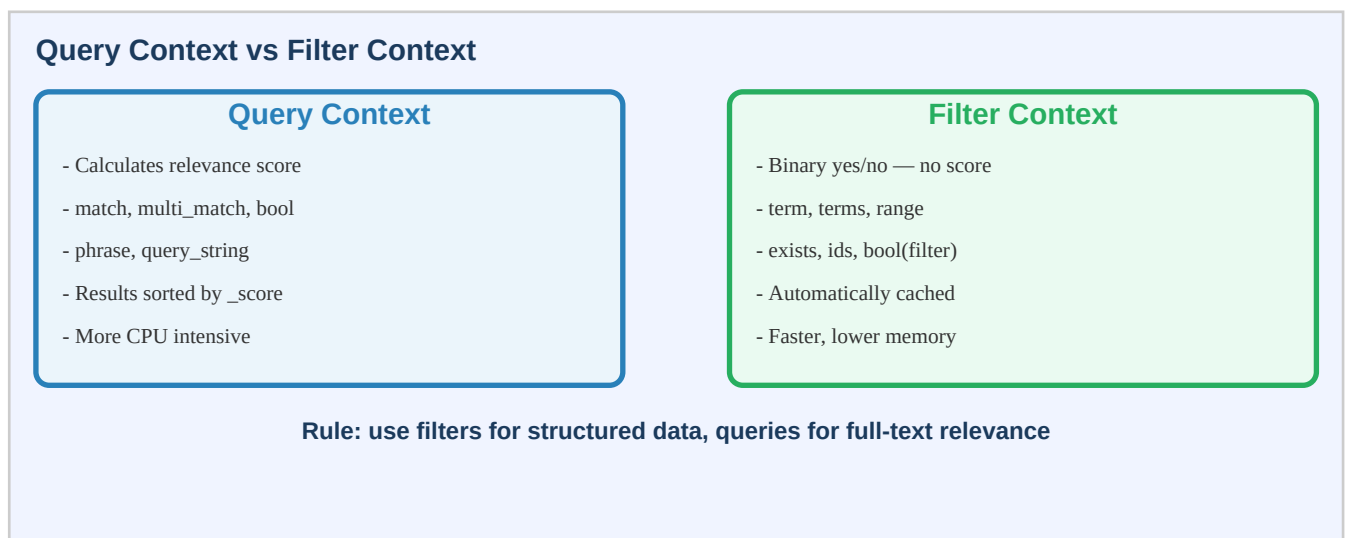
Chapter 10.4 – Elasticsearch Query DSL

Introduction

The Elasticsearch Query DSL (Domain Specific Language) is a JSON-based API for constructing complex search queries. It supports full-text search, structured filtering, geospatial queries, vector search, and analytical aggregations — all composable within a single JSON body. Understanding Query DSL is essential for any Java backend engineer integrating search into production services.

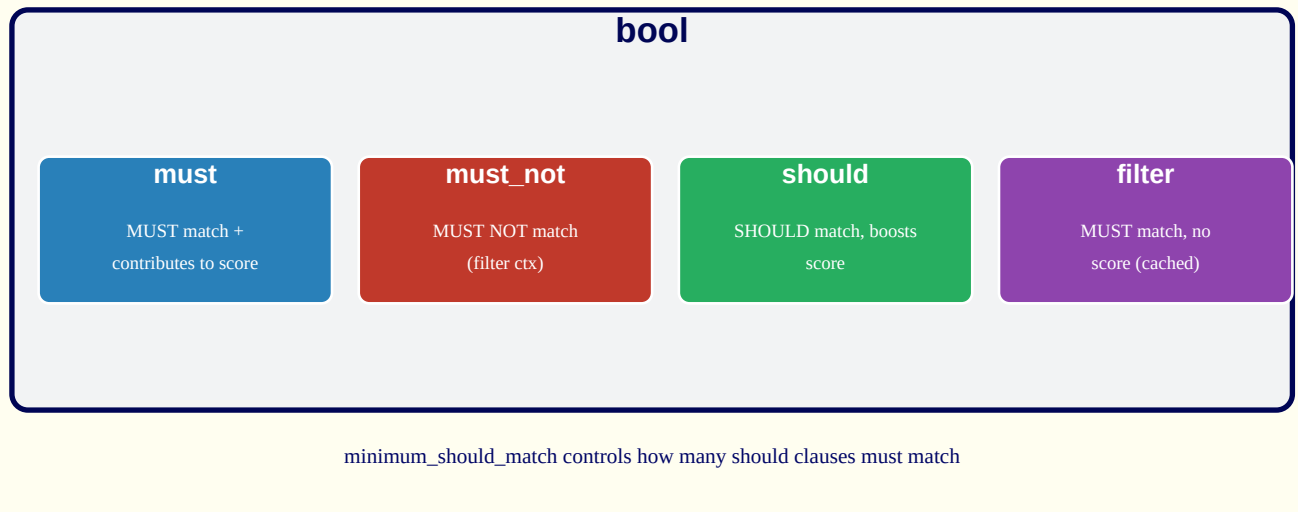
The two fundamental execution contexts are query context (computes relevance scores) and filter context (binary match, no scoring, cached). Choosing the correct context for each clause is the single most impactful performance decision in query design.

Query Context vs Filter Context



bool Query Structure

bool Query Structure



Query Types Overview

Common Query Types at a Glance

Full-Text	Term-Level	Compound	Geo/Vector
match	term	bool	geo_distance
multi_match	terms	boosting	geo_bounding_box
match_phrase	range	constant_score	knn
query_string	exists	dis_max	script_score
simple_query_string	ids	function_score	
	wildcard		

Full-text queries analyze the search string; term-level queries do not

Key Definitions

Query DSL	JSON API for expressing search queries, aggregations, and sorting in ES
query context	Clause contributes to relevance score (<code>_score</code>); used in <code>match</code> , <code>multi_match</code> , <code>bool.must</code>
filter context	Clause is binary (yes/no), adds no score, and is automatically cached
match query	Full-text search — analyzes the input and matches against analyzed text fields
term query	Exact-value search on keyword/numeric/date fields — NOT analyzed
bool query	Compound query combining <code>must</code> , <code>should</code> , <code>must_not</code> , <code>filter</code> clauses
multi_match	Runs a <code>match</code> query across multiple fields with optional per-field boosting
match_phrase	Searches for exact phrase — terms must appear in order with correct positions

range query	Finds documents where a field value is within a given range (gte, lte, gt, lt)
aggregation	Analytics on search results (terms, date_histogram, avg, sum, percentiles)
bucket aggregation	Groups documents into buckets (terms, range, date_histogram, nested)
metric aggregation	Computes a numeric metric over a bucket (avg, min, max, sum, cardinality)
pipeline aggregation	Operates on the output of other aggregations (moving_avg, derivative)
function_score	Modifies the relevance score using custom functions (field_value_factor, decay)
highlight	Returns matched fragments of text with surrounding context for display
source filtering	_source.includes/excludes to return only specific fields from _source

Query Type Reference

Query	Type	Analyzes Input?	Scored?	Use Case
match	Full-text	Yes	Yes	Search blog posts, descriptions
match_phrase	Full-text	Yes	Yes	Exact phrase, ordered terms
multi_match	Full-text	Yes	Yes	Search across name, desc, tags
query_string	Full-text	Yes	Yes	Power-user Lucene syntax
term	Term-level	No	Yes*	Exact keyword, filter by ID
terms	Term-level	No	Yes*	Match any of N exact values
range	Term-level	No	Yes*	Date/price range filter
exists	Term-level	No	No	Field has non-null value
bool	Compound	-	Yes	Combine multiple conditions
function_score	Compound	-	Yes	Custom scoring with decay/boost
constant_score	Compound	-	Fixed	Apply fixed score to filter

* When used in filter context (inside bool.filter), these produce no score.

Code Examples

1. Basic match and bool query

```
// POST /products/_search { "query": { "bool": { "must": [ { "match": { "name": "running shoes" } } ], "filter": [ { "term": { "category": "footwear" } }, { "range": { "price": { "gte": 50, "lte": 200 } } }, { "term": { "in_stock": true } } ], "should": [ { "term": { "tags": "sale" } } ], "minimum_should_match": 0 } }, "sort": [ { "_score": "desc" }, { "price": "asc" } ], "_source": ["id", "name", "price", "category"], "from": 0, "size": 20 }
```

2. multi_match with field boosting

```
// Boost name field 3x over description { "query": { "multi_match": { "query": "wireless bluetooth headphones", "fields": ["name^3", "description^1", "tags^2"], "type": "best_fields", // "most_fields", "cross_fields", "phrase" "operator": "and", "fuzziness": "AUTO" } } } //
```

```
cross_fields: treats all fields as one – good for name + surname // most_fields: runs match on
each field, sums scores // best_fields: uses score from best-matching field (default) //
phrase: runs match_phrase on each field
```

3. Aggregations — terms + date_histogram + avg

```
// Sales by category, avg price per month { "size": 0, "aggs": { "by_category": { "terms": {
"field": "category", "size": 10 }, "aggs": { "avg_price": { "avg": { "field": "price" } },
"monthly_sales": { "date_histogram": { "field": "created_at", "calendar_interval": "month" },
"aggs": { "revenue": { "sum": { "field": "price" } } } } } } }
```

4. Highlighting matched fragments

```
{ "query": { "match": { "description": "noise cancelling" } }, "highlight": { "fields": {
"description": { "fragment_size": 150, "number_of_fragments": 3, "pre_tags": ["<em>"],
"post_tags": ["</em>"] } } } } // Response includes "highlight": { "description": ["...great
<em>noise cancelling</em> ..."] }
```

5. function_score with decay

```
// Boost products created recently and with high ratings { "query": { "function_score": {
"query": { "match": { "name": "laptop" } }, "functions": [ { "gauss": { "created_at": {
"origin": "now", "scale": "30d", "decay": 0.5 } }, { "field_value_factor": { "field":
"rating", "factor": 1.2, "modifier": "sqrt", "missing": 1 } } ], "score_mode": "multiply",
"boost_mode": "multiply" } } }
```

6. Java: Elasticsearch Java Client (8.x) — bool query

```
import co.elastic.clients.elasticsearch.ElasticsearchClient; import
co.elastic.clients.elasticsearch._types.query_dsl.*; import
co.elastic.clients.elasticsearch.core.*; import
co.elastic.clients.elasticsearch.core.search.Hit; @Service public class ProductSearchService {
private final ElasticsearchClient esClient; public SearchResponse<ProductDocument>
search(String query, String category, double minPrice, double maxPrice, int page, int size)
throws IOException { return esClient.search(s -> s .index("products") .from(page * size)
.size(size) .query(q -> q .bool(b -> b .must(m -> m .match(mt -> mt .field("name")
.query(query) ) ) .filter(f -> f .term(t -> t .field("category") .value(category) ) ) .filter(f
-> f .range(r -> r .field("price") .gte(JsonData.of(minPrice)) .lte(JsonData.of(maxPrice)) ) )
) ) .sort(so -> so .field(f -> f .field("_score").order(SortOrder.Desc)) ) .highlight(h -> h
.fields("name", hf -> hf .numberOfFragments(0) .preTags("<em>") .postTags("</em>") ) ),
ProductDocument.class ); }
```

7. Paginating with search_after (deep pagination)

```
// Problem: from+size has 10,000 doc default limit (index.max_result_window) // Solution:
search_after with a sort tiebreaker // First page: { "size": 20, "sort": [{"created_at":
"desc"}, {"_id": "asc"}], "query": { "match_all": {} } } // Last hit's sort values:
["2024-01-15T10:00:00Z", "abc123"] // Next page uses search_after: { "size": 20, "sort":
[{"created_at": "desc"}, {"_id": "asc"}], "search_after": ["2024-01-15T10:00:00Z", "abc123"],
"query": { "match_all": {} } } // Java: SearchResponse<ProductDocument> response =
esClient.search(s -> s .index("products") .size(20) .sort(so -> so .field(f ->
f .field("created_at").order(SortOrder.Desc)) .sort(so -> so .field(f ->
f .field("_id").order(SortOrder.Asc)) ) .searchAfter(lastSortValues), // List<FieldValue>
ProductDocument.class );
```

8. Terms aggregation with cardinality

```
// Count distinct values with cardinality (approximate using HyperLogLog++) { "size": 0,
"aggs": { "unique_customers": { "cardinality": { "field": "customer_id",
"precision_threshold": 3000 // more = more accurate, more memory } }, "top_categories": {
"terms": { "field": "category", "size": 5, "order": { "revenue": "desc" } }, "aggs": {
"revenue": { "sum": { "field": "price" } } } } } }
```

Interview Questions & Answers

Q1. What is the difference between query context and filter context in ES?

Query context evaluates 'how well does this document match?' and computes a relevance score (`_score`). Filter context evaluates 'does this document match?' — a binary yes/no with no score calculation. Filters are automatically cached in the filter cache, making repeated filter queries much faster. Use filters for structured data (term, range, exists) and queries for full-text relevance (match, multi_match).

Q2. Explain the four clauses of a bool query.

1) ``must``: clauses that MUST match; contribute to the relevance score. 2) ``filter``: clauses that MUST match but run in filter context — no score contribution, cached. 3) ``should``: clauses that SHOULD match; matching ones boost the score. With no must/filter, at least one should must match (`minimum_should_match` defaults to 1). 4) ``must_not``: clauses that MUST NOT match; run in filter context (no score, cached).

Q3. When would you use match_phrase over match?

``match_phrase`` requires all terms to be present and in the same order with the same positional gap (slop defaults to 0). Use it for phrase searches like 'New York City' where word order matters. ``match`` allows any of the terms to match and ranks by frequency. Use ``match_phrase_prefix`` for as-you-type phrase completion. Phrase queries are more expensive as they use positional index data.

Q4. What is the difference between `term` and `match` queries?

``term`` does NOT analyze its input — it searches for the exact value as-is in the inverted index. Use it on ``keyword`` fields. If you use ``term`` on an analyzed ``text`` field, it likely won't match because the indexed terms are lowercase/stemmed. ``match`` analyzes the query string through the field's search analyzer before searching. Rule: term for keyword fields, match for text fields.

Q5. How do you perform a search across multiple fields?

Use ``multi_match``. The ``type`` parameter controls behavior: ``best_fields`` (default) scores by the best single field match; ``most_fields`` sums scores from all matching fields; ``cross_fields`` treats fields as one logical field (good for person name split across first_name/last_name); ``phrase`` runs `match_phrase` on each field. Use ``^`` notation for per-field boosting, e.g., ``fields: ['title^3', 'body']``.

Q6. What is the purpose of `minimum\_should\_match` in a bool query?

``minimum_should_match`` defines how many ``should`` clauses must match. It accepts absolute integers (e.g., 2 = at least 2 should clauses must match) or percentages ('75%'). When a bool query has only ``should`` clauses and no ``must`` or ``filter``, `minimum_should_match` defaults to 1. When there are must/filter clauses, it defaults to 0 (should clauses only boost score). Set it to control precision vs. recall.

Q7. Explain how the `terms` aggregation works and its limitations.

``terms`` aggregation groups documents by unique field values and returns the top N buckets with their document counts. Limitation: in a sharded environment, each shard returns its own top N; the coordinator

merges these. This means results can be approximate — a term that ranks 6th on shard A and 4th on shard B might be missed. Increase ``shard_size`` (default: `size * 1.5 + 10`) to reduce error at the cost of more data transfer.

Q8. What is ``source filtering`` and why use it?

Source filtering controls which fields are returned from ``_source`` in search results. Use ``_source: [field1,field2]`` (shorthand) or ``_source: {includes: [...], excludes: [...]}``. Benefits: reduces network transfer and deserialization cost, especially when ``_source`` contains large fields (binary data, long descriptions). Always filter ``_source`` to only what the client needs, particularly in high-traffic APIs.

Q9. How do you handle deep pagination in Elasticsearch?

Three approaches: (1) ``from + size`` — simple but limited to 10,000 docs by default (`index.max_result_window`) and expensive as from increases (ES scores all docs up to `from+size`). (2) ``search_after`` — stateless cursor using the sort values of the last hit; efficient for sequential pagination; requires a consistent sort with a tiebreaker (`_id`). (3) Scroll API — deprecated in 8.x; use Point In Time (PIT) + `search_after` for snapshots.

Q10. What is the Point In Time (PIT) API?

PIT creates a lightweight consistent snapshot of the index state, identified by a `pit_id`. You can then paginate with `search_after` against that frozen view — even as documents are added/deleted/updated, the PIT ensures consistent results. Create: ``POST /index/_pit?keep_alive=5m``. Use the `pit_id` in search requests. Delete when done: ``DELETE /_pit``. Introduced in ES 7.10 to replace the Scroll API.

Q11. What is ``function_score`` and how do you use decay functions?

``function_score`` wraps a base query and modifies scores using custom functions. Decay functions (gauss, exp, linear) reduce the score as a numeric/date field moves away from an origin. Example: boost products by recency using a Gaussian decay on ``created_at`` with a 30-day scale. ``field_value_factor`` multiplies the score by a field value (e.g., rating). ``random_score`` adds randomness. ``score_mode`` controls how multiple function scores combine (multiply, sum, avg, max).

Q12. Explain the ``constant_score`` query.

``constant_score`` wraps a filter and returns all matching documents with a fixed ``_score`` of 1.0 (or a specified ``boost`` value). Since it uses filter context internally, it's cached. Use it when you want all matching documents to appear equally (e.g., 'show all in-stock items first regardless of relevance') or to apply a fixed boost to a filter clause inside a `function_score` query.

Q13. What is ``dis_max`` query and when would you use it?

``dis_max`` (disjunction max) returns the maximum score among its sub-queries rather than summing them. Use it when querying the same text against multiple fields where the best match should dominate: searching 'python' against both ``title`` and ``body`` — a strong title match shouldn't be watered down by a weaker body match. ``tie_breaker`` (0 to 1) adds a fraction of other sub-query scores. ``multi_match`` with ``type: best_fields`` uses `dis_max` internally.

Q14. How do you use the scroll API and why is it deprecated?

Scroll maintains a search context on each shard, allowing retrieval of all matching documents beyond the 10k limit: create with ``POST /index/_search?scroll=1m``, iterate using the ``_scroll_id``. Deprecated because it holds shard resources (memory, file handles) for the scroll lifetime, creating cluster pressure under many concurrent scrolls. Replacement: PIT + `search_after`, which is stateless and doesn't hold shard resources.

Q15. Explain bucket vs metric vs pipeline aggregations.

Bucket aggregations group documents into sets (buckets): ``terms`` (by value), ``date_histogram`` (by time), ``range``, ``nested``. Each bucket can have sub-aggregations. Metric aggregations compute a single value over a set of documents: ``avg``, ``sum``, ``min``, ``max``, ``cardinality``, ``percentiles``. Pipeline aggregations operate on the OUTPUT of other aggregations (not documents): ``moving_avg``, ``derivative``, ``bucket_sort``, ``cumulative_sum``. They reference sibling/parent aggregations by path.

Q16. What is a ``cardinality`` aggregation and how accurate is it?

``cardinality`` computes the approximate count of distinct values using the HyperLogLog++ algorithm. It's approximate — the error rate is typically under 5% with default settings. The ``precision_threshold`` parameter (1–40,000, default 3000) trades memory for accuracy: values below the threshold are almost exact. Use for analytics like 'unique users today' where a 2-5% error is acceptable. For exact counts, use scripted `value_count` (expensive).

Q17. How does ES execute aggregations differently from queries?

Queries work on the inverted index and are optimized for finding matching documents quickly. Aggregations predominantly work on ``doc_values`` (column-oriented on-disk structures), which are efficient for iterating over all values of a field. This is why aggregating on ``text`` fields (which lack `doc_values`) requires enabling ``fielddata`` (heap-expensive) or adding a ``keyword`` sub-field. Aggregations always scan all matching documents, so narrow filters applied before aggregation are critical for performance.

Q18. What is the ``highlight`` API and how does it work?

Highlighting returns snippets of text from matched documents with the query terms marked. ES supports three highlighters: ``unified`` (default, uses BM25 over sentence boundaries), ``plain`` (Lucene's original, uses positional index), and ``fvh`` (fast vector highlighter — requires ``term_vector``: `with_positions_offsets`` at index time). Configure ``fragment_size``, ``number_of_fragments``, ``pre_tags``, and ``post_tags``. Set ``number_of_fragments``: 0 to highlight the entire field.

Q19. What is the ``explain`` API and when do you use it?

``GET /index/_explain/docId`` with a query body explains why a specific document did or did not match, and how its score was computed. It shows the full BM25 breakdown: IDF, TF, field norm contribution for each term. Add ``?explain=true`` to a ``_search`` request to get explanations for all hits. Use it to debug: why is doc A ranked above doc B? why is my term query not matching? It's for debugging only — never in production code.

Q20. Explain ``index.max_result_window`` and the risks of increasing it.

``max_result_window`` (default 10,000) limits ``from + size``. Increasing it allows deeper pagination but is dangerous: ES must score and sort ALL documents up to ``from + size`` in memory on every shard, then merge. For `from=100,000, size=10`: each shard loads 100,010 docs into heap. With many shards this causes GC pressure and OOM errors. Instead, use `search_after` + PIT for deep pagination.

Q21. What is ``_reindex`` and how does it differ from a direct copy?

``_reindex`` copies documents from a source index to a destination index using ES's internal scroll+bulk mechanism. It: (1) respects the destination mapping (re-analyzes text), (2) optionally transforms documents via a script, (3) handles version conflicts per policy. It is NOT a byte copy — every document is re-indexed from scratch. Use it for mapping changes, analyzer changes, index splits, or cross-cluster migration (with ``remote`` source). Monitor with tasks API: ``GET /_tasks?actions=*reindex``.

Q22. How does ``query_string`` differ from ``simple_query_string``?

``query_string`` supports full Lucene syntax: AND/OR/NOT, field:value, wildcards, fuzzy (~), ranges ([a TO b]), boosting (^). It throws an error on invalid syntax. Use for power-user search where users control the query. ``simple_query_string`` is more forgiving — invalid syntax is silently ignored rather than causing an error. Better for user-facing search boxes. Both parse the query string into a compound Lucene query.

Q23. What is a ``nested`` query and when is it needed?

When a field is mapped as ``nested``, each array element is stored as a separate hidden document. A regular query would incorrectly merge values across elements (e.g., matching `'color:red'` from element 1 with `'size:XL'` from element 2). A ``nested`` query evaluates conditions within the scope of a single nested document, giving accurate per-element matching. Required for: order line items, review collections, variant options.

Q24. What is a ``has_child`` / ``has_parent`` query?

``has_child`` returns parent documents that have matching child documents. ``has_parent`` returns child documents whose parent matches. These work with parent-child join field relationships (where parent and child share the same shard). Use case: blog posts and comments — find all posts that have at least one approved comment. Compared to nested, join relationships allow parent/child to be updated independently; nested requires reindexing the parent to update a nested element.

Q25. How do you implement a faceted search UI with Elasticsearch?

Use aggregations alongside queries: (1) Run your main query. (2) Add a terms aggregation for each facet dimension (category, brand, color). (3) Use ``post_filter`` to apply facet selections AFTER aggregations — so facet counts reflect global distribution, not the filtered set. (4) Use ``global`` aggregation to compute facet counts independent of the main query for sidebar filters. This is the standard e-commerce faceted search pattern.

Q26. Describe ``post_filter`` and how it differs from a regular filter.

``post_filter`` is applied AFTER query execution and aggregation computation. This means aggregations reflect the unfiltered result set, while the returned hits are filtered. Example: user filters by `'color:red'` — the hits show only red items, but category/brand aggregations still count all colors. Without `post_filter`, the aggregations would only count red items, breaking the sidebar facet display. Use `post_filter` only for faceted navigation; use regular filter for actual query constraints.

Q27. What is the ``profile`` API?

``profile: true`` in a search request returns detailed timing information about every component of query execution: how long each Lucene query took, how long aggregations took per shard, and which operations were the most expensive. It returns a JSON structure with shards, query breakdown (create, next_doc, advance, match, score times), and collection timings. Use it to identify which query clause is slow. Never use in production requests — it significantly increases response size and processing time.

Q28. How do you implement spell-check / did-you-mean in ES?

Use the Suggesters API: (1) ``term`` suggester: suggests similar terms for each token independently using edit distance. (2) ``phrase`` suggester: corrects entire phrases; better for 'did you mean' suggestions. (3) ``completion`` suggester: for autocomplete. For phrase suggester, specify ``collate`` to verify that suggested corrections actually return results. The suggester works on the inverted index of the specified field — use a ``keyword`` field or text field with appropriate settings.

Q29. How do you sort by multiple fields in ES?

Pass an array to ``sort``: ``[{'_score': 'desc'}, {'price': 'asc'}, {'_id': 'asc'}]``. ES sorts by each criterion sequentially — ties at the first level are broken by the second. For ``_score`` to be meaningful, include it first. For keyword/numeric fields, sort uses ``doc_values``. For text fields (no `doc_values` by default), enable ``fielddata`` or use a ``keyword`` sub-field. Always include ``_id`` as a tiebreaker for stable pagination.

Q30. What are ``indices_boost`` and field-level ``boost`` and how do they differ?

``indices_boost`` applies a multiplier to the scores of all documents from specific indices in a cross-index search. Field-level ``boost`` (via ``^`` in `multi_match` or ``boost`` parameter) multiplies the relevance contribution of

a specific field at query time. Both affect the final `_score` but at different levels. Note: boost values modify BM25 scores, so very high boosts can overshadow other relevance factors — calibrate empirically with real queries.

Q31. What is a ``script`` query and what are its performance implications?

A ``script`` query executes a Painless script against every document in the result set to determine matching. It bypasses the inverted index — ES must evaluate the script per document. This is very expensive at scale. Use only when no DSL query can express the condition. Prefer runtime fields (precompile scripts at mapping time) or ``script_score`` only over already-filtered result sets to minimize the doc count the script runs against.

Q32. Explain the ``collapse`` feature in Elasticsearch.

Field collapsing returns only the top-ranked document per unique value of a field, while reporting the total number of groups (via ``total_hits``). Example: return the best result per vendor from a product search. Syntax: ``collapse: {field: 'vendor_id', inner_hits: {name: 'top3', size: 3}}`. ``inner_hits`` returns the top N docs per group. Useful for deduplication in search UIs. Note: total hits reflects pre-collapse count.

Q33. What is the ``geo_distance`` query and how do you use it with sorting?

``geo_distance`` filters documents within a specified radius from a `geo_point`: ``{geo_distance: {'distance': '10km', 'location': {'lat': 40.7, 'lon': -74.0}}}``. Combine with ``_geo_distance`` sort to rank results nearest-first: ``sort: [{_geo_distance: {'location': {'lat': 40.7, 'lon': -74.0}, 'order': 'asc', 'unit': 'km'}}]``. The ``location`` field must be mapped as ``geo_point``. Use ``geo_bounding_box`` for rectangular area queries and ``geo_polygon`` for arbitrary shapes.